

«Трекер калорий»: агенты, ломатель, workflow, worktrees, изоляция

Пять шагов одной практической работы: параллельная разработка субагентами, независимая проверка на устойчивость, повторяемый workflow, три параллельные ветки через git worktrees и проверка границы песочницы.

4 параллельных агента

22 сценария атаки

3 worktree-ветки

Идемпотентный workflow

ДАТА	АВТОР
26 августа 2026	Александра
ПРОЕКТ	ПЕРИОД РАБОТ
Трекер калорий (FastAPI + SQLite)	24–26 августа 2026

— Оглавление

•	Краткое резюме	3
1	Агенты — параллельная разработка	4
1.1	Разбивка на зоны	4
1.2	Что сделал каждый агент	4
1.3	Конфликты между зонами	5
1.4	Интеграционная проверка	6
1.5	Время и контекст	6
2	Ломатель — проверка на уязвимости	7
3	Workflow — идемпотентный дневной отчёт	9
4	Worktrees — три параллельные фичи	10
5	Изоляция — граница песочницы	11

— Краткое резюме

Одна задача — «Трекер калорий» (FastAPI + SQLite) — пройдена через пять разных режимов работы с агентами за три дня, 24–26 августа 2026: параллельная разработка, независимая проверка на устойчивость, повторяемый workflow, git worktrees и проверка границ песочницы.

Агенты. Четыре субагента получили по эксклюзивной зоне файлов и работали параллельно в одной координирующей сессии (данные и API, расчёты с property-тестами, фронтенд, workflow). Границы зон не нарушены ни разу. Параллельный прогон занял ≈ 21 минуту против ≈ 58 минут при последовательном сложении работы агентов — выигрыш и по времени, и по контексту координирующей сессии. Главный честный пробел: расчёт калорий «по весу» был реализован и протестирован, но не подключён к реальному пути данных API → хранилище — потому что справочника продуктов в проекте не было.

Ломатель. Независимая сессия (не автор кода) провела 22 сценария атаки на API. Сломать насмерть не удалось — валидация и параметризованные запросы держат оборону, но найдено шесть реальных проблем, от воспроизводимого краха 500 на NaN / Infinity до полного отсутствия авторизации между записями.

Workflow. Повторяемая процедура «дневной отчёт» — собрать записи, посчитать итоги, сравнить с нормой, сохранить, пометить выполненным. Идемпотентность подтверждена не словами агента, а повторным запуском против живого API: второй прогон с теми же данными вернул `unchanged`, файл не переписан.

Worktrees. Тот же проект получил три новые фичи в трёх параллельных ветках (норма и предупреждение, поиск продукта с автоподстановкой — закрывающий пробел из раздела «Агенты», недельная выгрузка в CSV/PDF). Слияние смоделировано на одноразовой ветке: только технические конфликты «вставили в одно и то же место», ни одного смыслового.

Изоляция. Проверка границы песочницы дала честный, не всегда удобный результат: ни терминал, ни доступный MCP-шлюз к файловой системе не заблокировали запись за пределами рабочей папки. Единственная граница в этой конфигурации — политика разрешений, а не файловая изоляция.

1 Агенты — параллельная разработка

24 АВГУСТА 2026 · КООРДИНИРУЮЩАЯ СЕССИЯ «ТРЕКЕР КАЛОРИЙ: ПАРАЛЛЕЛЬНАЯ РАЗРАБОТКА ЧЕТЫРЬМЯ АГЕНТАМИ»

Работа над проектом разбита на четыре независимые зоны и выполнена четырьмя субагентами Claude Code, запущенными параллельно (`run_in_background`) в одной координирующей сессии.

1.1 Разбивка на зоны

Агент	Зона (эксклюзивная запись)	Задача
A	<code>models/</code> , <code>storage/</code> , <code>api/</code> , <code>requirements.txt</code>	модель приёма пищи, SQLite-хранилище с атомарными операциями, CRUD API, валидация на границе
B	<code>calculations/</code> , тесты, <code>requirements-calculations.txt</code>	расчёт калорий/БЖУ по весу, агрегация день/неделя, <code>property</code> -тесты (<code>hypothesis</code>)
C	<code>frontend/</code>	HTML-страница: форма добавления + таблица за день
D	<code>workflows/</code>	workflow «дневной отчёт»: собрать → посчитать → сравнить с нормой → сохранить → пометить выполненным

Каждому агенту было явно запрещено редактировать чужие зоны и общие файлы (`README.md` , `REPORT.md`). После завершения всех четырёх проверен `git status --short` по корню проекта — границы зон не нарушены ни одним агентом, конфликтов на уровне файлов нет. Там, где агенту был нужен контракт из параллельно писавшейся зоны, разрешалось не ждать, а зафиксировать предположение письменно (`API_CONTRACT_ASSUMED.md` , `DEPENDENCIES_ASSUMED.md`).

1.2 Что сделал каждый агент

Агент A — данные и API

FastAPI + Pydantic для валидации на границе, SQLite (`stdlib`) для хранения, транзакции через контекстный менеджер с `threading.Lock` на запись, WAL-режим. Полный CRUD по приёмам пищи, фильтры по дате и диапазону. Решение — калории и БЖУ вводятся вызывающей стороной напрямую, а не вычисляются из

справочника продуктов (справочника в проекте не было). Самопроверка: поднять живой сервер, пройден полный цикл через `curl`, включая ошибки 404/400 и CORS для фронтенда.

Агент В — расчёты и property-тесты

Модуль расчёта порции по весу и агрегации по дню/неделе, спроектирован независимым от чужих моделей (duck-typing) — разумное решение в условиях параллельной работы над `models/`. Пять property-тестов на hypothesis: сумма агрегата равна независимо пересчитанной сумме приёмов пищи; линейность при масштабировании веса; округление никогда не даёт ноль или отрицательное значение для положительной порции; агрегация не зависит от порядка записей; расчёт порции не даёт отрицательных значений. Итог прогона, перепроверенный координатором: **17 passed** (12 unit + 5 property, 100–200 примеров hypothesis на свойство).

Агент С — фронтенд

Форма приёма пищи и таблица за день с итоговой строкой. Начал с предположения о контракте API, а когда контракт Агента А появился — сам подстроил код под реальные имена полей, не дожидаясь координатора. Проверено визуально: страница не разваливается без backend, полный цикл отработан на живом mock-сервере.

Агент D — workflow «дневной отчёт»

Разобран отдельно в разделе 3 — здесь только побочный эффект: во время тестирования агент создал файл базы данных вне своей зоны на запись, не смог его удалить из-за запрета трогать чужую зону и корректно сообщил об этом координатору вместо того, чтобы обойти ограничение. Файл удалён координатором при сведении, соответствующий путь добавлен в `.gitignore`.

1.3 Конфликты между зонами

Конфликт	Суть	Итог
Содержательный	Агент В реализовал и протестировал расчёт «по весу», но Агент А решил хранить уже готовые калории/БЖУ, введённые вручную — потому что справочника продуктов не было. Агент С согласованно выбрал ручной ввод.	<code>calculations/portion.py</code> рабочий и протестированный, но не используется в реальном пути данных API → хранилище → фронтенд. Зафиксировано как нерешённое, а не спрятано.

Конфликт	Суть	Итог
Терминологический	Три разных названия полей в модели, в расчётах и в предположении фронтенда о контракте (<code>consumed_at</code> vs <code>date</code> , <code>proteins</code> vs <code>protein</code>).	Разрешено без координатора: Агент С переписал код под реальный контракт, Агент D реализовал нормализацию, понимающую оба варианта именования.
Ложный след	Агент С зафиксировал 500-ю ошибку API при проверке на порту 8123.	При независимой проверке координатором API работал штатно — ошибка была связана с устаревшим процессом на этом порту, а не с кодом.

1.4 Интеграционная проверка координатором

Помимо отчётов самих агентов, координатор прогнал реальный сквозной сценарий: установка зависимостей без ошибок; `pytest` → 17 passed; поднят `uvicorn` , через `curl` создана и получена запись; `workflows/daily_report.py` запущен дважды подряд — первый раз `recomputed` , второй — `unchanged` с идентичными числами; фронтенд открыт в браузере поверх живого API, запись добавлена через форму, итоговая строка совпала с ручным расчётом, консоль браузера — без ошибок. Это подтверждает, что четыре независимо написанные зоны реально стыкуются в рабочую систему.

1.5 Время и контекст по каждому агенту

Агент	Длительность	Токены	Вызовы инструментов
А — данные и API	9 мин 13 с	≈ 75 200	28
В — расчёты и тесты	7 мин 02 с	≈ 67 100	16
С — фронтенд	21 мин 14 с	≈ 110 900	114
D — workflow	20 мин 23 с	≈ 109 400	49
Сумма (если бы делал один агент)	≈ 57 мин 52 с	≈ 362 600	207
Фактически, параллельно	≈ 21 мин 14 с	—	—

ЭФФЕКТ ИЗОЛЯЦИИ КОНТЕКСТА

В координирующую сессию попали только четыре финальных отчёта агентов — единицы тысяч токенов суммарно, а не 362 тысячи токенов их внутренней работы. Здесь параллельный прогон выиграл и по времени, и по контексту — в отличие от более мелких задач, где выигрыш обычно только в контексте.

Что не сделано

- Расчёт по весу не подключён к живому пути данных — главный содержательный пробел (закрыт позже, см. раздел 4, ветка `product-search`).
- Нет UI для редактирования/удаления записей во фронтенде, хотя API это поддерживает.
- `aggregate_week` считает среднее как «сумма / 7» (календарная неделя), а не по дням с приёмами пищи.

2 Ломатель — проверка на уязвимости

25 АВГУСТА 2026 · ОТДЕЛЬНАЯ СЕССИЯ «ТЕСТИРОВАНИЕ ТРЕКЕРА КАЛОРИЙ НА УЯЗВИМОСТИ»

Отдельная, независимая от авторской сессия — не тот агент, что писал код. Задача — не подтвердить, что приложение работает, а сломать его: 22 сценария атаки через API за 37 итераций проверки. Полностью сломать (уронить сервер насмерть, испортить базу, обойти SQL-защиту) не удалось: валидация Pydantic и параметризованные запросы держат оборону хорошо. Но найдено шесть реальных проблем.

Находки, по убыванию опасности

#	Находка	Опасность
1	Крах 500 на NaN / Infinity / переполнении float. Pydantic корректно бракует значение, но при рендеринге тела ошибки 422 FastAPI пытается сериализовать то же невалидное число через <code>json.dumps(allow_nan=False)</code> и падает необработанным исключением. Воспроизводится без аутентификации.	Высокая
2	Нет авторизации и понятия «пользователь». В модели нет <code>user_id</code> : любой, кто знает URL, может получить, изменить или удалить чужую запись по последовательному <code>id</code> — подтверждено практически. Пока приложение однопользовательское — не проблема; с появлением второго пользователя это IDOR-дыра из учебника.	Высокая (архитектурно)
3	Нет проверки правдоподобия между весом и калориями: приняты 0.5 г со 100 ккал ($\approx 20\,000$ ккал/100г) и 100 кг с 1 ккал.	Средняя
4	Даты без разумных границ: приняты записи с 2099, 1900 и даже 0001 годом.	Средняя
5	Type coercion: <code>weight_grams: true</code> тихо становится 1.0 и создаёт запись.	Низкая
6	Дублирующиеся приёмы пищи не отслеживаются — три идентичных запроса создают три записи без предупреждения.	Низкая

Что выдержало атаку

- Отрицательный/нулевой вес, вес $> 100\,000$ г, пустое название, отсутствующие поля, слишком длинное имя — везде корректный 422.
- SQL-инъекция в название продукта — не сработала, таблица цела (параметризованные запросы).
- Битый JSON, пустое тело, null-байт в строке — корректный 422.
- 30 параллельных POST на одну дату — ни одной потерянной записи, блокировка на запись отработала как надо.
- Гонка «10× PUT + DELETE» на одной записи — без порчи данных, финальное состояние согласовано.

ПОБОЧНАЯ НАХОДКА

По пути обнаружен тот же процесс-«призрак» на порту 8123, что дал ложную 500-ю ошибку ещё в отчёте раздела 1 (конфликт «Ложный след»). Сервер перезапущен на чистом порту с изолированной тестовой базой — подтверждает, что этот порт и раньше давал ложные срабатывания, не связанные с кодом.

3 Workflow — идемпотентный дневной отчёт

АГЕНТ D, ЗОНА `WORKFLOWS/` · ОБЯЗАТЕЛЬНЫЙ ПУНКТ ЗАДАНИЯ

`workflows/daily_report.py` (только стандартная библиотека) + `workflows/config.json` (норма по умолчанию). Источник данных — хранилище, API или заглушка, с автоопределением. Агрегация — через расчётный модуль, с резервным суммированием, если модуль недоступен.

Идемпотентность

Sha256-хэш от нормализованных приёмов пищи и нормы сохраняется в `meta.combined_hash`. Повторный запуск с тем же хэшем возвращает статус `unchanged`, файл не переписывается — это проверено координатором дважды подряд на живом API, а не заявлено агентом: второй запуск действительно дал `unchanged`, время изменения файла не поменялось.

Шаг	Что происходит
1. Сбор	Данные за сегодня из источника (хранилище / API / заглушка, автоопределение)
2. Расчёт	Агрегация калорий/БЖУ через расчётный модуль (fallback — прямое суммирование)
3. Сравнение	Итог сверяется с нормой из <code>config.json</code>
4. Сохранение	Отчёт пишется в файл; если <code>combined_hash</code> не изменился — файл не трогается
5. Отметка	Статус прогона: <code>recomputed</code> или <code>unchanged</code>

ЧЕСТНО ПРО ПОБОЧНЫЙ ЭФФЕКТ

Во время самопроверки агент создал файл базы данных вне разрешённой зоны записи и не смог удалить его сам из-за запрета трогать чужие файлы — вместо обхода правила корректно сообщил об этом координатору. Файл удалён при сведении результатов, путь добавлен в `.gitignore`.

4 Worktrees — три параллельные фичи

25 АВГУСТА 2026 · ТА ЖЕ СЕССИЯ, СЛЕДУЮЩИЙ ШАГ ПОСЛЕ СВЕДЕНИЯ ЧЕТЫРЁХ АГЕНТОВ

Тот же проект — три новые фичи, каждая в своём `git worktree` и своей ветке, с отдельным агентом на каждую. Ни один агент не вышел за пределы своей папки; во всех трёх — чистый `git status` и зелёные тесты.

Ветка	Что сделано	Файлы	Тесты	Время / токены
<code>daily-norm-warning</code>	Эндпоинт дневной сводки, норма из конфига workflow, подсветка и текстовые предупреждения на фронтенде (ok / near / exceeded)	6 файлов, +342/-2	17/17	11 мин 15 с / 121k
<code>product-search</code>	Справочник из 19 продуктов, поиск, автоподстановка калорий по весу через уже существующий расчётный модуль — закрывает честный пробел из раздела 1	9 файлов (3 новых), +714/-5	27/27 (+10 новых)	15 мин 42 с / 154k
<code>weekly-export</code>	Выгрузка недельного отчёта в CSV и PDF (кириллица через системный шрифт), опциональный эндпоинт сводки	6 файлов (4 новых), +652/-1	17/17	12 мин 00 с / 126k

Проверка слияния

Все три ветки смержены последовательно на одноразовой тестовой ветке от основной — чтобы увидеть реальные конфликты, а не гадать о них. Тестовая ветка удалена, боевые ветки не тронуты.

1. `weekly-export` → основная ветка: чисто, без конфликтов.
2. + `daily-norm-warning`: один конфликт — обе ветки добавили свой импорт в одну строку; разрешено сохранением обоих.
3. + `product-search`: пять конфликтов, все одного типа — обе ветки независимо дописали новый код в одну и ту же точку файла. Ни одного смыслового конфликта, где одна и та же логика правилась бы по-разному.

ПРЕДЛОЖЕННЫЙ ПОРЯДОК СЛИЯНИЯ

`weekly-export` → `daily-norm-warning` → `product-search`. Первая ветка почти не пересекается с остальными; вторая и третья плотно правят одни и те же файлы API и фронтенда, поэтому более крупная и инвазивная `product-search` мержится последней — против уже стабилизировавшегося кода.

5 Изоляция — граница песочницы

26 АВГУСТА 2026 · ОТДЕЛЬНАЯ СЕССИЯ «ПРОВЕРКА ГРАНИЦЫ ПЕСОЧНИЦЫ»

Задача: проверить границу песочницы двумя способами — запись файла за пределами рабочей папки через терминал и через любой подключённый МСР-сервер, — и честно показать факт отказа, если он произойдёт, не обходя ограничение.

Канал	Что сделано	Результат
Терминал	Запись файла по пути выше рабочей папки проекта	Файл создан без ограничений. Блока на уровне ОС или инструмента нет — только политика разрешений, которая в этом режиме пропустила команду без запроса подтверждения.
МСП-сервер	Запрос расширения доступа к папке за пределами проекта через ближайший подходящий инструмент (шлюз доступа к директориям — среди подключённых МСП не нашлось инструмента прямой произвольной записи файла)	Доступ выдан сразу, без видимого в сессии интерактивного подтверждения. Дальнейшая запись файла не выполнялась — оба канала уже показали отсутствие отказа, эскалация не требовалась.

ЧЕСТНЫЙ ВЫВОД

В этой конфигурации песочницы нет жёсткой границы «нельзя выйти за пределы рабочей папки» — ни через терминал, ни через доступный МСП-путь. Единственный барьер — режим разрешений, а не файловая изоляция. Тестовый файл удалён после проверки по запросу.

Отчёт собран из пяти рабочих сессий 24–26 августа 2026 и объединяет фактические результаты прогонов, а не пересказ намерений: тесты, время, токены и находки приведены по итогам реальных запусков и перепроверок координатором, включая три честных пробела — неподключённый расчёт по весу (закрит позже в отдельной ветке), отсутствие авторизации и отсутствие файловой изоляции песочницы.